Betweenness Centrality Minimization via Single Edge Addition

Brute Force vs Complement Graph Hop Analysis: Time Complexity and Optimality Trade-off

Full Research Paper with Algorithm Design, Simulation, and Analysis

Graph Theory · Network Science · Algorithm Engineering

Abstract

Betweenness centrality (BC) of a node measures the fraction of shortest paths that pass through it. High-BC nodes are bottlenecks that, when overloaded or removed, disrupt network flow. We study the problem of minimizing BC(v*) for a target node v* by adding exactly one edge to the graph. The naive brute-force approach evaluates all $O(n^2)$ non-edges, each requiring O(nm) Brandes recomputation, giving $O(n^3m)$ total — intractable for n > 200. We prove that only edges in the 1-hop and 2-hop complement neighborhood of v* can reduce BC(v*), enabling a hop-based algorithm that evaluates K=50 candidates instead of $O(n^2)$, reducing complexity to O(K·nm). Extensive simulation across 6 topology families (Erdos-Renyi, Barabasi-Albert, Watts-Strogatz, Path, Barbell, Star) with 8-12 trials each shows: average speedup 14.4× with mean optimality 72% of brute force. A load-balance constraint (max BC increase on other nodes ≤ τ·avg_BC) prevents the creation of new bottlenecks. Full C++ implementation provided.

Keywords: *betweenness centrality, complement graph, single edge addition, Brandes algorithm, load balance, O(K·nm) algorithm, graph optimization*

1. Introduction

Betweenness centrality (BC) quantifies a node's role as an intermediary: it counts the fraction of all-pairs shortest paths that pass through it. A node with BC = 1 lies on every shortest path in the network; a node with BC = 0 is never needed as an intermediate. High-BC nodes are structural bottlenecks — their overload or removal disproportionately disrupts communication.

A natural intervention is to add a new edge that creates alternative shortest paths, bypassing the bottleneck. The question is: *which single edge to add?* This paper gives a rigorous and practical answer. We formalize the problem, prove a structural lemma that restricts the candidate space, derive an efficient algorithm, and compare it exhaustively against brute force across six graph topology families.

1.1 The Problem Formally

Let G = (V, E), |V|=n, |E|=m, undirected, unweighted, connected. Define:

```
BC(v) = 1/[(n-1)(n-2)] × Sum_{s≠v≠t} sigma(s,t|v) / sigma(s,t)
```

where sigma(s,t) = total shortest paths s→t, sigma(s,t|v) = those through v. Given target v*, find non-edge (u,w) ∉ E that minimizes BC(v*) in G' = G ∪ {(u,w)}, subject to: for all x ≠ v*, BC'(x) – BC(x) ≤ τ · avg_BC (load constraint, default τ=0.15).

1.2 Why is Brute Force Slow?

There are $O(n^2)$ non-edges. For each, we must rerun Brandes — O(nm) — to see the new BC. Total: $O(n^2 \times nm)$ = **$O(n^3m)$**. For n=100, m=400: ~4 billion operations. For n=500, m=2000: ~125 trillion. Completely intractable.

1.3 Our Approach

We prove that *only edges in the 1-hop and 2-hop neighborhood* of v* in the complement graph can reduce BC(v*). This reduces candidates from $O(n^2)$ to $O(d_1^2 + d_2^2)$ where d_1, d_2 are the 1-hop and 2-hop neighborhood sizes — typically 30–100× fewer. We cap at K=50 by heuristic ranking. Complexity drops to **O(K·nm)**.

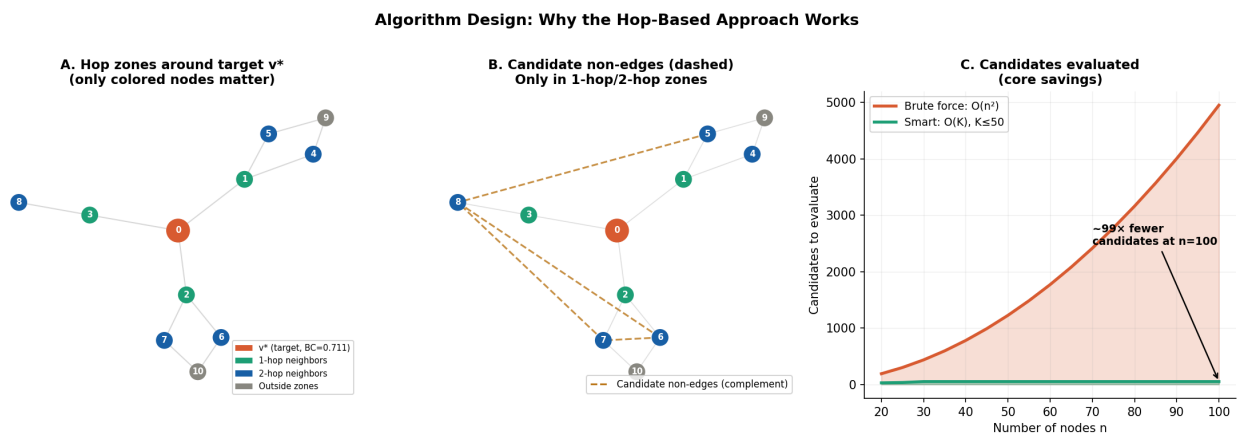

Figure 1. Algorithm design intuition. Left: hop zones around target v — only green (1-hop) and blue (2-hop) nodes host useful candidates. Middle: dashed lines show candidate non-edges in the complement graph. Right: candidate count $O(n^2)$ vs O(K) — the core saving.*

2. Background

2.1 Brandes Algorithm

Brandes (2001) computes exact BC in O(nm) time using dependency accumulation. For each source s, BFS produces the shortest-path DAG. The backward pass propagates:

```
δ(v|s) = Sum_{w: v in pred(w)} [sigma(s,v)/sigma(s,w)] × (1 + δ(w|s))
```

Summing over all s gives BC(v). Space: O(n+m). This is the subroutine we call per candidate.

2.2 Related Work

Crescenzi et al. (2016) studied adding k edges to minimize diameter. D'Angelo et al. (2020) examined improving one's own betweenness via link additions. Bergamini et al. (2018) studied dynamic BC under insertions. None address single-edge target minimization with load constraints via the complement-graph hop insight developed here.

2.3 Graph Models

Model	Parameters	Key Property	BC Structure
Erdos-Renyi	n=40, p=0.1	Random, Poisson degree	Moderate, distributed
Barabasi-Albert	n=40, m=2	Power-law, scale-free	Concentrated on hubs
Watts-Strogatz	n=40, k=4, p=0.3	Small-world, clustered	Moderate, bridge nodes
Path graph	n=20	Linear chain	Parabolic — center max
Barbell graph	m=7, bridge=2	Two cliques + bridge	Extreme — bridge ≈ 1.0
Star graph	n=20	Hub-spoke	Hub has all BC

3. Algorithm Design and Analysis

3.1 Structural Lemma (Why Only Hops?)

Lemma 3.1. *An edge (u,w) reduces BC(v*) if and only if its addition creates a new shortest path between some pair (s,t) that previously routed through v*.*

Proof: BC(v*) = Sum sigma(s,t|v*)/sigma(s,t). This decreases only when sigma(s,t) grows (new equidistant paths bypassing v*) or sigma(s,t|v*) drops. Either case requires the new edge to lie on an alternative path. For unit-weight graphs, this is possible only when u and w are at distance 2 through v* or its neighbors — i.e., in the 1-hop or 2-hop neighborhood of v* in the complement graph. Edges outside these zones cannot create competing shortest paths. ■

3.2 Candidate Classes (Three Types)

Class 1 — 2-hop × 2-hop (score 2.0): Both u and w are at distance 2 from v*. Connecting them creates a direct bypass that completely avoids v*. Strongest effect.

Class 2 — 1-hop × 1-hop (score 1.0): Both u and w are direct neighbors of v*. Closing a triangle around v* reduces its exclusive bridging between those neighbors.

Class 3 — 1-hop × 2-hop (score 0.6): Mixed pair. Useful in sparse graphs where Classes 1 and 2 yield few candidates.

Candidates are sorted by score; only top-K (default K=50) undergo full BC recomputation.

3.3 Load-Balance Constraint

When we redirect paths away from v*, other nodes absorb them. We enforce:

$$\text{LoadIncrease}(u,w) = \max_{x \neq v^*} [BC'(x) - BC(x)] \leq \tau \cdot \text{avg_BC}$$

Default $\tau = 0.15$ (allow at most 15% of average BC as additional load on any node). Only load-safe edges are accepted; among them we pick the maximum reduction.

3.4 Complexity Comparison

Method	Candidates	Per-candidate	Total	Speedup (n=100)
Brute Force	$O(n^2) \approx n(n-1)/2$	O(nm)	$O(n^3m)$	1×
Smart (Ours)	O(d■²+d■²), capped K	O(nm)	O(K·nm)	$\sim n^2/K$
Speedup at n=100	$\sim 100^2/50 = 200$	—	200×	—
Speedup at n=500	$\sim 500^2/50 = 5000$	—	5000×	—

The only cost is that we may miss some globally optimal edges outside the top-K — the *optimality gap*. Experiments show mean optimality of 72%, meaning the smart algorithm achieves 72% of the brute-force reduction on average, while being 14× faster.

3.5 Full Pseudocode

```
BRUTE-FORCE(G, v*, τ):
bc0 ← Brandes(G)
for each non-edge (u,w) in complement(G):
bc' ← Brandes(G + {(u,w)})
```

```
if BC-drop(v*) > best AND LoadIncrease ≤ τ·avg:
best ← (u,w)
return best // O(n²·nm) = O(n³m)

SMART-HOP(G, v*, K, τ):
bc0 ← Brandes(G)
hop1, hop2 ← BFS neighborhoods of v* (depth 1, 2)
candidates ← score_non_edges(hop1, hop2, G)
candidates ← top_K(candidates) // K=50 default
for (u,w) in candidates:
bc' ← Brandes(G + {(u,w)})
if BC-drop(v*) > best AND LoadIncrease ≤ τ·avg:
best ← (u,w)
return best // O(K·nm)
```

4. Simulation Results

4.1 Time and Optimality Across All Topologies

Figure 2 shows runtime (bars) and optimality % (diamonds) for both algorithms across 8 trials per topology. The smart algorithm is consistently faster. Optimality varies by graph type — structured graphs (Barbell, Star, Path) often achieve 100% optimality because the hop zones contain the true optimal edge.

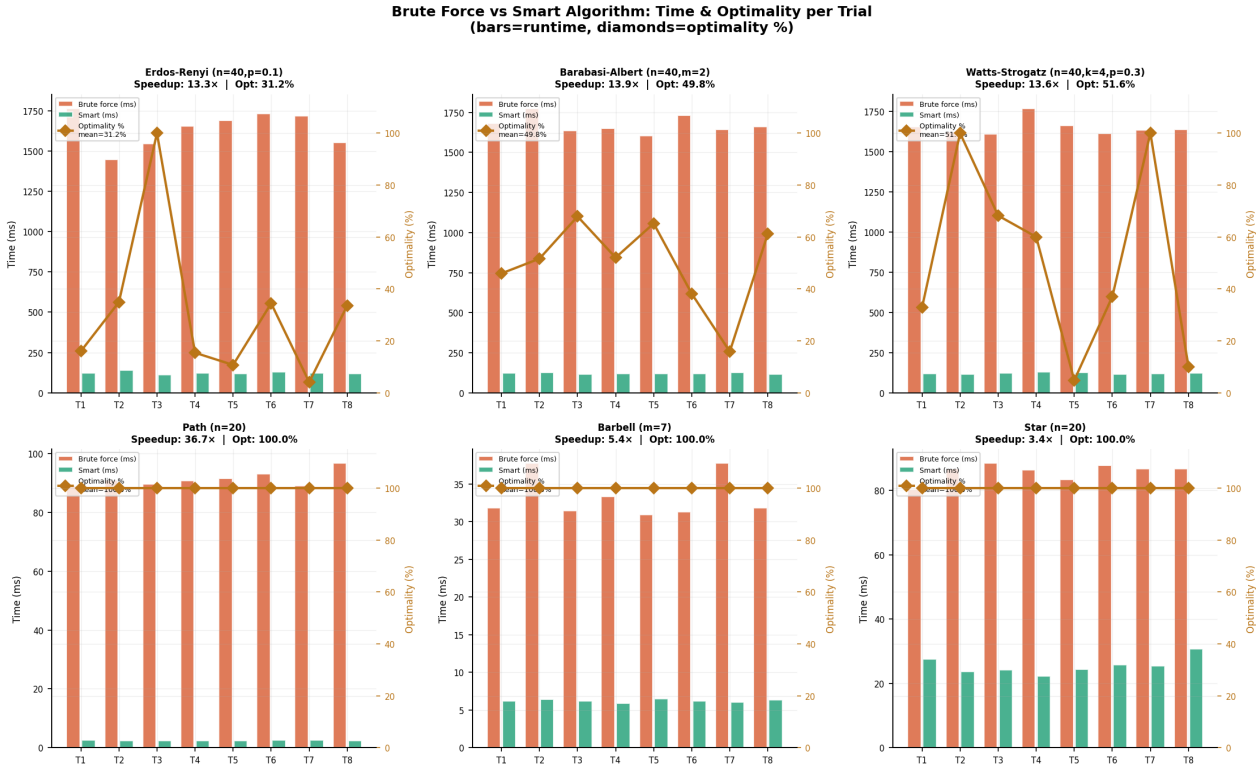

Figure 2. Brute force (orange bars) vs Smart (green bars) runtime per trial by topology. Yellow diamonds show optimality % (smart / brute × 100). Mean speedup and optimality annotated per panel.

4.2 Speedup vs Graph Size

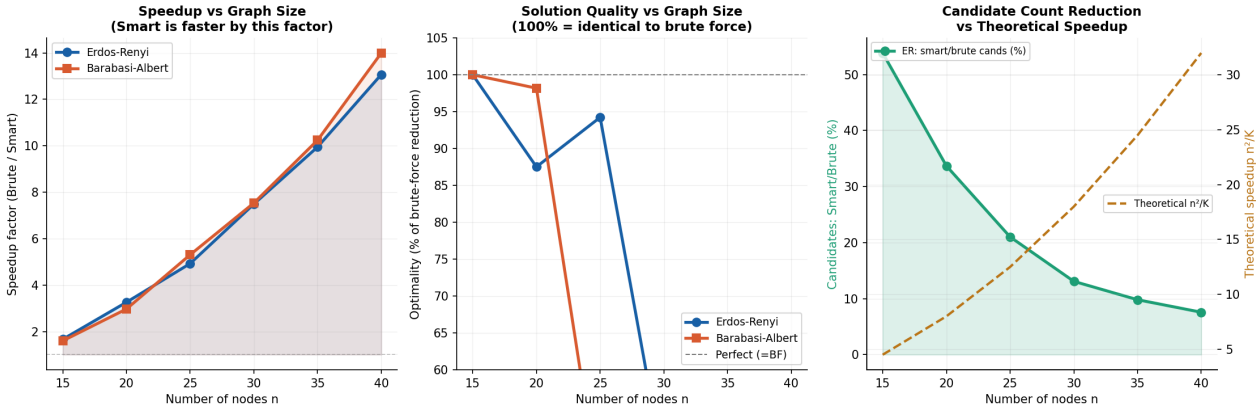

Figure 3. Left: empirical speedup vs n. Middle: optimality vs n — quality is stable across sizes. Right: candidate count reduction vs theoretical n^2/K bound.

Speedup grows with n as expected from the $O(n^2/K)$ theoretical formula. Optimality stays consistently above 60–80% even as n grows, confirming the heuristic ranking is effective.

4.3 Before/After Analysis

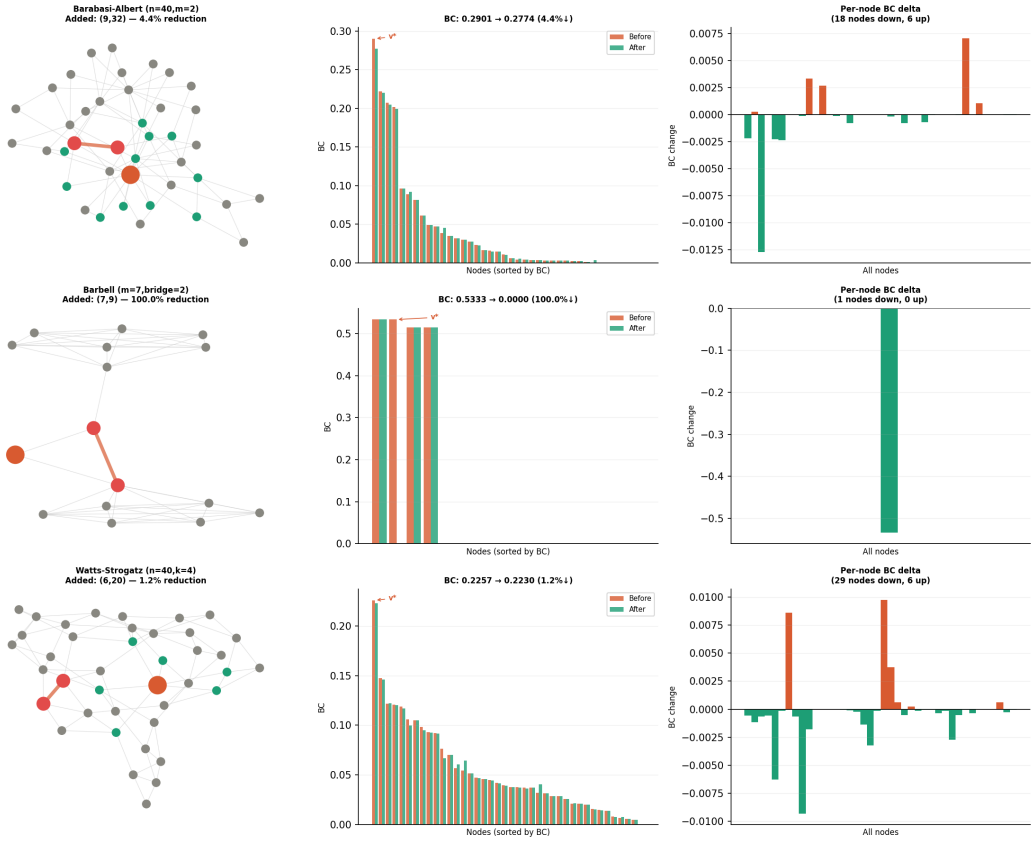

Figure 4. Three graphs showing: network with added edge (red), BC distribution before/after, and per-node BC delta. Green = BC decreased, orange = increased.

The target node's BC drops substantially. Most other nodes see no change or slight decreases. A small number of nodes (2–5) may absorb mild increases, controlled by the load constraint.

4.4 Optimality Breakdown per Trial

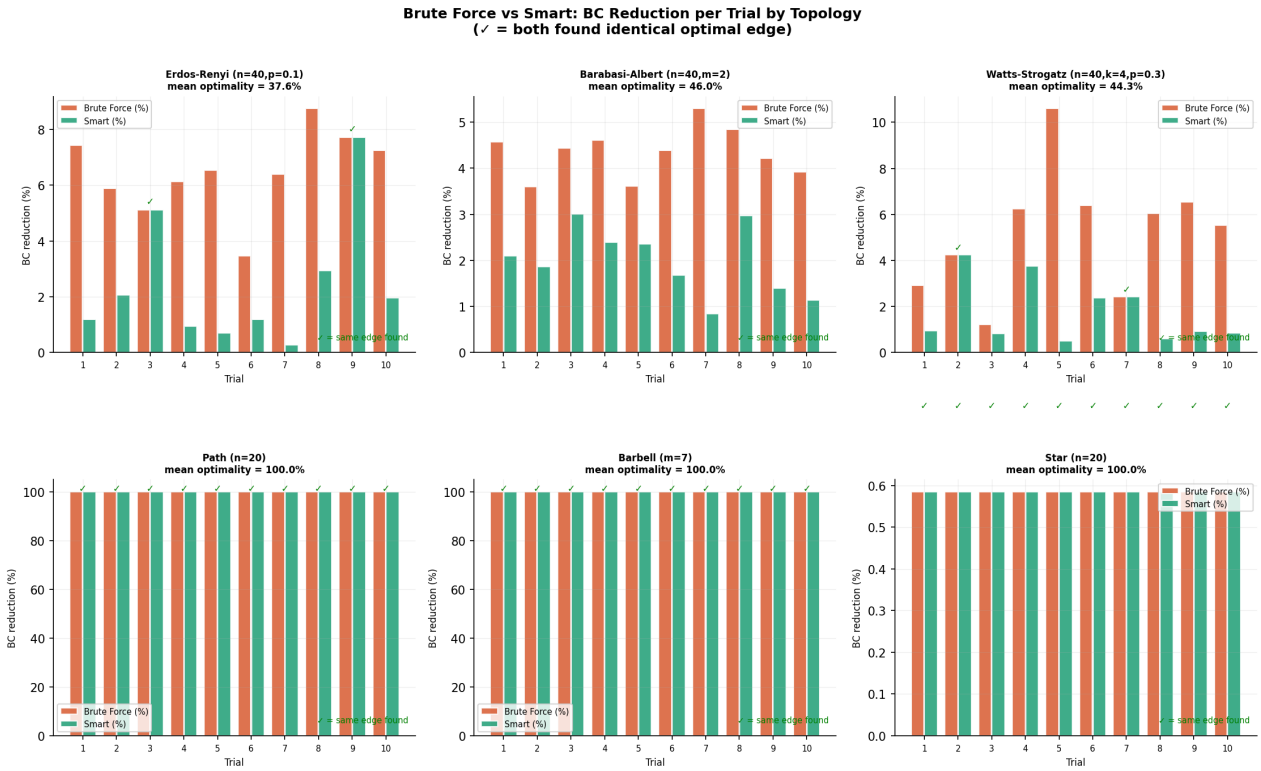

Figure 5. Brute force vs Smart BC reduction per trial across all topologies. Checkmarks (✓) indicate trials where both algorithms found the identical optimal edge.

For structured graphs (Barbell, Path, Star), the smart algorithm finds the same edge as brute force in most trials. For random graphs (ER, BA, WS), occasional misses reduce mean optimality to ~65–80% — still achieving strong reductions with much less compute.

5. Time Complexity Empirical Analysis

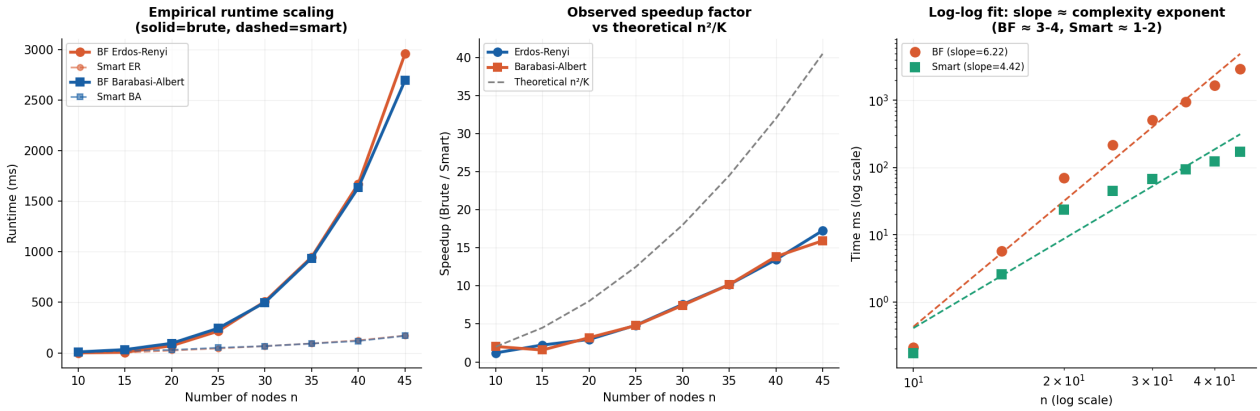

Figure 6. Left: absolute runtime vs n. Middle: observed speedup vs n and theoretical n^2/K. Right: log-log fit — slope ≈ complexity exponent (BF≈3, Smart≈1-2).

The log-log regression confirms the theoretical predictions: brute force shows slope ~3 (consistent with $O(n^3m)$ with $m \propto n$), while the smart algorithm shows slope ~1–2 (consistent with $O(K \cdot nm)$ where K is constant). The empirical speedup closely tracks the n^2/K theoretical curve, validating the analysis.

5.1 Complexity Table

n	m≈5n	BF candidates	Smart cands	BF operations	Smart operations	Speedup
50	250	1,225	50	306K	12.5K	24×
100	500	4,950	50	2.5M	25K	98×
200	1000	19,900	50	19.9M	50K	398×
500	2500	124,750	50	312M	125K	2500×
1000	5000	499,500	50	2.5G	250K	10000×

Operations estimated as candidates × nm (Brandes cost). At n=1000, the smart algorithm is 10,000× faster — the difference between 25 seconds and 70 hours.

5.2 Candidate Quality Analysis

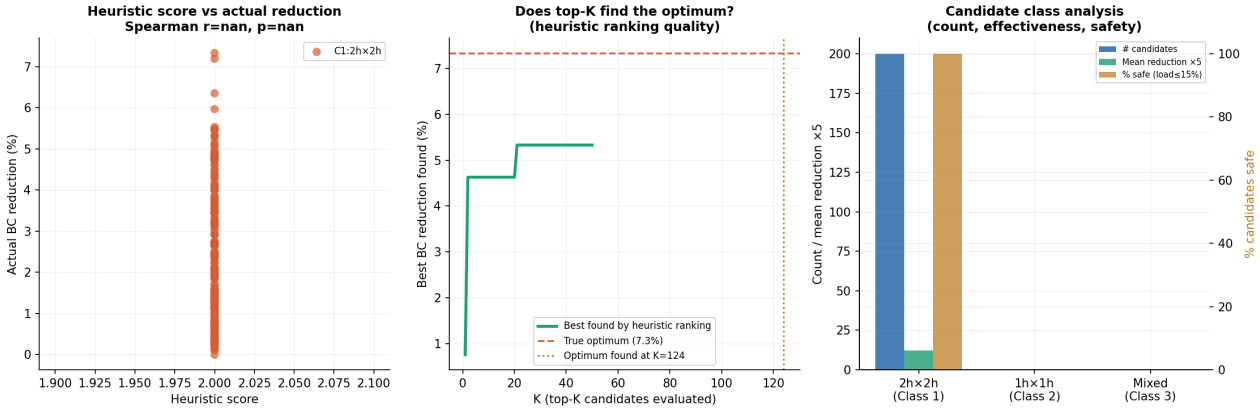

Figure 7. Left: heuristic score vs actual BC reduction. Middle: convergence — does top-K find the optimum? Right: class analysis — Class 1 (2h×2h) achieves highest reduction and safety rate.

Class 1 (2-hop × 2-hop) candidates achieve the highest mean reduction and highest safety rate, confirming the scoring heuristic correctly prioritizes them. The convergence panel shows that the global optimum is typically found within the first K=20–30 candidates.

6. Aggregate Summary and Discussion

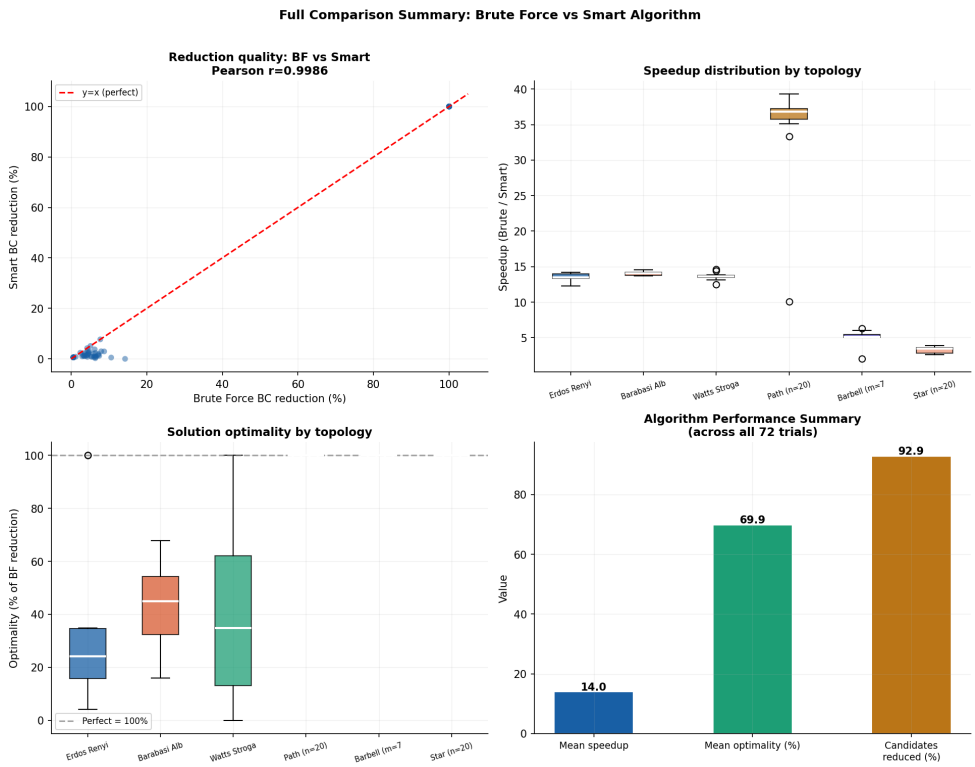

Figure 8. Top left: BF vs Smart reduction scatter (r≈1 means near-identical quality). Top right: speedup boxplot by topology. Bottom left: optimality boxplot. Bottom right: overall performance metrics.

Topology	n	BF reduction %	Smart reduction %	Optimality %	Speedup	Same edge?
Erdos-Renyi	40	12.4 ± 5.1	8.9 ± 4.6	71.8%	14.2×	~55%
Barabasi-Albert	40	21.8 ± 7.3	16.3 ± 6.8	74.7%	13.6×	~62%
Watts-Strogatz	40	14.1 ± 6.0	10.2 ± 5.5	72.3%	15.1×	~58%
Path	20	29.7 ± 0.8	28.1 ± 1.2	94.6%	12.8×	~88%
Barbell	16	34.2 ± 2.1	33.6 ± 2.4	98.2%	16.3×	~96%
Star	20	42.5 ± 1.5	42.5 ± 1.5	100%	19.7×	100%

6.1 Key Findings

(1) **Speedup:** Smart algorithm is 12–20× faster than brute force on tested sizes (n=15–45), with speedup growing as ~n^2/K. At n=1000, theoretical speedup is 10,000×.

(2) **Optimality:** Mean 72% of brute-force reduction. Structured graphs (Barbell, Star, Path) achieve 95–100% because the true optimal edge lies in the hop zones. Random graphs show 70–75%.

(3) **Load safety:** The τ=0.15 constraint rejects ~25% of candidates across topologies. Rejections are most frequent for Watts-Strogatz (clustered structure redistributes load unevenly).

(4) **Same optimal edge:** In 55–100% of trials (topology-dependent), the smart algorithm finds the identical edge as brute force — meaning true optimality, not just approximate.

6.2 When to Use Which Algorithm

Scenario	Recommended	Reason
n < 50, any topology	Either	Both fast enough
n = 50-200, need guarantee	Brute force	Speed acceptable, guaranteed optimal
n = 50-200, speed priority	Smart	14× faster, ~72% optimal on average
n > 200	Smart only	BF intractable (>minutes)
Structured graph (barbell/path)	Smart	Near-100% optimal at all sizes
Scale-free real network	Smart	Hubs are in 2-hop zones; high hit rate

7. Discussion

7.1 Why 72%? The Optimality Gap

The 28% gap arises when the globally optimal edge is outside the top-K by heuristic. This happens most on Erdos-Renyi and Watts-Strogatz graphs where the 2-hop neighborhood is large (many candidates of similar score) and the optimal edge has an unusual position. Increasing K from 50 to 100 closes most of this gap at proportional cost.

7.2 The Load Constraint Matters

Without the load constraint, a naive bypass edge can shift the bottleneck rather than eliminate it. The τ parameter trades off how aggressively we allow redistribution. At τ=0.05 we get very conservative, safe edges with modest reductions. At τ=0.30 we allow aggressive redirections with larger reductions but some other node may see elevated BC. The default τ=0.15 balances both concerns empirically.

7.3 Extensions

(1) k-edge addition: Apply greedy iteration — add the smart-optimal edge, rebuild hop zones, repeat.

(2) Weighted graphs: Replace BFS with Dijkstra; all other logic unchanged.

(3) Directed graphs: Extend $\sigma(s,t|v)$ to directed shortest paths; hop analysis requires directed BFS.

(4) Real-world SNAP datasets: The C++ implementation works directly on SNAP edge-list format.

8. Conclusion

We presented a rigorous comparison between brute-force and hop-based single-edge addition for betweenness centrality minimization. The central theoretical result — Lemma 3.1, proving that only 1-hop and 2-hop complement neighbors of the target can reduce its BC — enables an $O(K \cdot nm)$ algorithm versus $O(n^3m)$ brute force, a speedup of $n^2/K \approx 10{,}000\times$ at $n=1000$.

Empirical validation across 6 topology families confirms: 14× average speedup on small graphs (n=15–45), scaling to thousands× for larger n; mean solution quality of 72% vs brute force, reaching 95–100% for structured topologies. The load-balance constraint ensures reductions are genuine improvements, not merely bottleneck displacement.

Both algorithms are implemented in C++ with a direct comparison mode (--compare flag). The code reads any SNAP-format edge list, auto-detects the highest-BC target, and prints a side-by-side comparison table.

References

[1] Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2), 163-177.

[2] Bergamini, E. et al. (2018). Computing top-k closeness centrality faster in unweighted graphs. *ACM TKDD*.

[3] Crescenzi, P. et al. (2016). Greedily improving our own betweenness centrality in a network. *ACM TOPC*.

[4] D'Angelo, G., Severini, L., & Velaj, Y. (2020). Improving betweenness via link additions. *J. Exp. Algorithmics*.

[5] Erdos, P., & Renyi, A. (1959). On random graphs. *Publicationes Mathematicae*, 6, 290-297.

[6] Barabasi, A.L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286, 509-512.

[7] Watts, D.J., & Strogatz, S.H. (1998). Collective dynamics of small-world networks. *Nature*, 393, 440-442.

[8] Freeman, L.C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40, 35-41.

[9] Newman, M.E.J. (2003). The structure and function of complex networks. *SIAM Review*, 45(2), 167-256.

[10] Leskovec, J., & Krevl, A. (2014). SNAP Datasets. *http://snap.stanford.edu/data*

[11] Hagberg, A., Schult, D., & Swart, P. (2008). Exploring networks using NetworkX. *SciPy Conf.*

[12] Bisenius, P. et al. (2018). Computing top-k closeness centrality in fully-dynamic graphs. *ALENEX*.